ITM UNIVERSITY GWALIOR

School of Computing and Intelligent Systems

PROJECT-BASED LEARNING (PBL)

MINI PROJECT REPORT

Sports Match Outcome Pattern Analysis Using Python

Project detail	Information
Course	Data Analytics Using Python (CSL-0312)
Program	B.Tech. CSE (Artificial Intelligence & Machine Learning)
Semester / Section	III / F
Academic Session	July–December 2026
Domain	Sports Analytics / Cricket Match Analysis
Team Leader	G Jaswanth Reddy (BETM1AI25159)
Team Members	K Manoj Kumar (BETN1AI25183); K Punith kumar (BETN1AI25176)
Faculty Mentor	Dr. Sanjay Jain
Submission date	15/10/2026

1. Introduction & Problem Statement

Cricket matches produce structured information such as competing teams, toss results, venue, innings scores, match winners, and win margins. Looking at individual matches alone can make broader patterns difficult to identify. Python-based data analytics can organize these records, summarize match outcomes, and visualize patterns across teams, seasons, and match conditions.

The main problem addressed by this project is how to identify recurring patterns associated with cricket match outcomes. The analysis examines team win counts and win rates, toss outcomes, batting-first versus chasing results, and winning margins. Where the dataset supports it, venue and season can also be compared.

The project is exploratory. It can describe associations in the available match records, but it cannot prove that winning the toss, choosing to bat or bowl, or playing at a particular venue causes a win. Team strength, player availability, weather, match format, and other factors may influence outcomes.

Project objectives

• Load and inspect a structured cricket match dataset using Pandas.

• Clean missing, duplicated, and inconsistent match records.

• Summarize wins by team and season.

• Compare match outcomes after toss wins and between batting first and chasing.

• Use charts and basic statistics to communicate observed patterns.

• Document limitations and suggest improvements for future analysis.

Scope

The initial scope is limited to match-level outcome analysis. Ball-by-ball modelling, player-level prediction, and live forecasting are optional extensions and are not claimed as completed in this report.

2. Dataset Description & Source

The report template's student dataset is replaced here with a cricket match dataset. No sports CSV was attached with the source report, so the exact record count, column names, missing-value counts, and statistical results must be filled in after the selected dataset is loaded. A match-level IPL dataset is suitable for the proposed analysis.

Item	Description
Dataset name	Cricket / IPL Match Results Dataset
Expected file	matches.csv (use the actual downloaded filename)
Potential sources	Kaggle IPL datasets; Cricsheet; official IPL statistics
Unit of analysis	One row per match
Key fields	season, date, team1, team2, toss_winner, toss_decision, winner, venue, result, result_margin
Target/outcome field	winner
Quality checks	Inspect nulls, duplicate match records, team-name variants, and abandoned/no-result matches

Dataset CSV: Match_Info.csv (GitHub)

Attribute overview

• season / date: identifies when a match took place.

• team1 / team2: the competing teams.

• toss_winner / toss_decision: toss result and the elected action.

• winner: match winner; may be blank for no-result matches.

• venue: location where the match was played.

• result / result_margin: type and size of victory, where available.

3. Data Cleaning & Preprocessing

The CSV should be loaded with Pandas and inspected before analysis. Standardize column names, remove exact duplicate rows only after checking match identifiers, parse dates if available, and inspect missing values. Rows with no winner should be handled separately rather than automatically assigned to a team. Team-name variants should be harmonized carefully so the same team is not counted under multiple labels.

Representative Python code

```
import pandas as pd

df = pd.read_csv("matches.csv")
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
print("Missing values:\\n", df.isna().sum())
print("Exact duplicate rows:", df.duplicated().sum())

# Normalize column labels for easier referencing
df.columns = df.columns.str.strip().str.lower()
df = df.drop_duplicates()

# Check outcome fields before analysis
if "winner" in df.columns:
    print("Missing winners:", df["winner"].isna().sum())
if "season" in df.columns:
    print(df["season"].value_counts().sort_index())
```

Preprocessing decisions

• Check missing values in winner, toss_winner, season, and venue.

• Exclude no-result matches from win-rate denominators, or report them as a separate outcome category.

• Use the actual number of eligible matches as the denominator for each rate.

• Keep a record of cleaning steps so the results can be reproduced.

4. Exploratory Data Analysis (EDA)

EDA describes the shape and coverage of the match dataset before drawing conclusions. Useful summaries include the number of matches per season, number of teams, team win counts, frequency of toss decisions, and distribution of winning margins. For numerical fields, use descriptive statistics only where those fields are present and meaningful.

Recommended summary table

Measure	How to calculate / interpret
Total records	Number of rows in the source file
Eligible matches	Matches with a valid winner and required fields
Unique teams	Distinct normalized team names
Season coverage	Earliest and latest season in the file
Toss decision share	Count and percentage for bat / field choices
Win margin	Summary of runs or wickets, keeping units separate

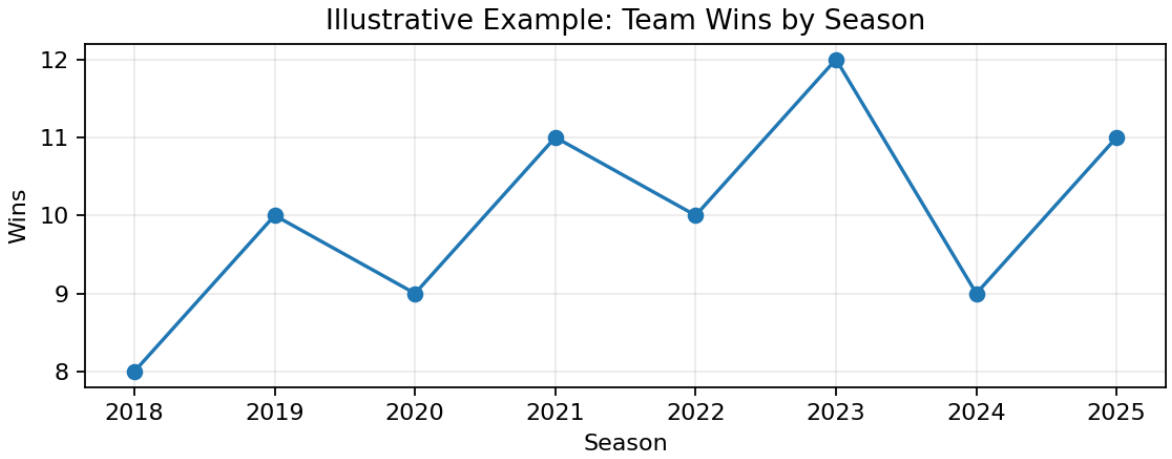


Figure 1. Illustrative example of wins by season. The values are dummy values to demonstrate the chart format and must not be interpreted as actual IPL results.

Suggested interpretation: after running the analysis, identify seasons with increases or decreases in wins for a selected team. Consider changes in competition size and season format before comparing raw win counts across years.

5. Data Visualization & Observations

Visualizations make differences and distributions easier to interpret. Use counts and percentages together where possible, because raw totals can be misleading when teams play different numbers of matches.

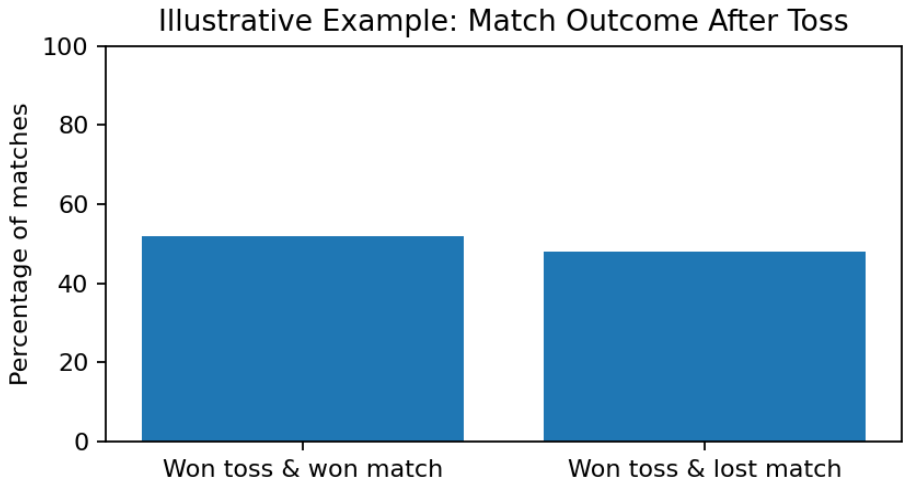


Figure 2. Illustrative toss outcome comparison. Replace the example percentages with the proportion of eligible matches in the selected dataset.

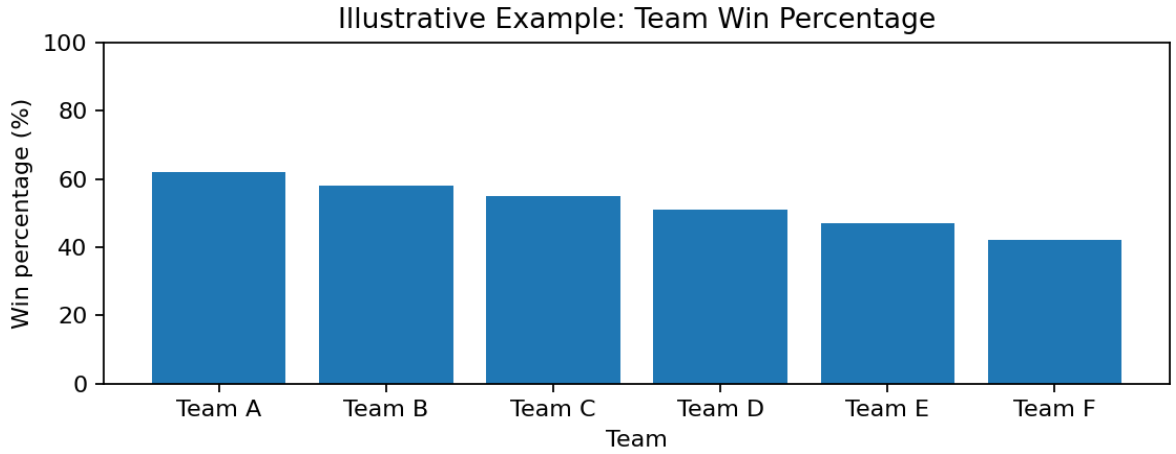


Figure 3. Illustrative team win percentage comparison. Calculate each team's win percentage as wins divided by eligible matches played, multiplied by 100.

Observation prompts

• Does the toss winner also win the match more often than not? Report both count and percentage. • Which teams have the highest win percentage, and did they play comparable numbers of matches? • Do patterns change by season or venue?

6. Outcome Pattern Analysis & Basic Statistics

Team win rate

For each team, calculate matches played as the count of eligible matches in which the team appears as team1 or team2. Calculate wins as the count of those matches where winner equals the team. Win rate = wins / matches played × 100. Validate that no-result matches are treated consistently.

Toss and match outcome

Compare the number and percentage of matches where the toss winner also won the match. This is a descriptive comparison only. A chi-square test may be considered for categorical variables if its assumptions are met, but statistical significance does not establish causation.

Batting first versus chasing

If the dataset contains a reliable winner and toss decision, a simple comparison can be made between matches where the toss winner elected to field and whether the toss winner won. To compare batting-first and chasing wins directly, the dataset needs a reliable innings outcome or winner-by-innings indicator. Do not infer batting order from toss decision alone.

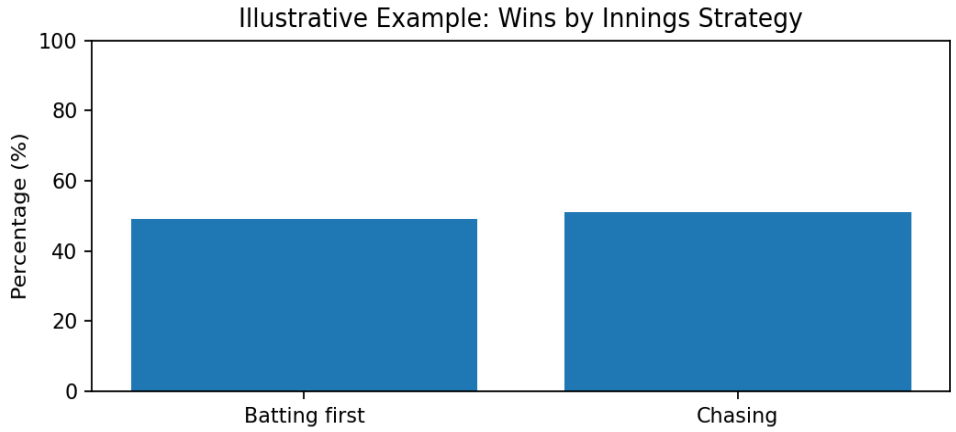


Figure 4. Illustrative batting-first versus chasing comparison; example values are not actual match statistics.

Example Python calculation

```
# Adapt column names to your CSV
valid = df.dropna(subset=["winner", "team1", "team2"]).copy()
teams = pd.concat([valid["team1"], valid["team2"]]).value_counts()
wins = valid["winner"].value_counts()
win_summary = pd.DataFrame({"matches_played": teams,
                            "wins": wins}).fillna(0)
win_summary["win_rate_pct"] = (
    win_summary["wins"] / win_summary["matches_played"] * 100
)
print(win_summary.sort_values("win_rate_pct", ascending=False))
```

7. Key Insights & Interpretation

The final insights should be written only after running the code on the chosen dataset. Use the following structure to report results without overstating what the data shows.

Results table to complete

Question	Result to insert after analysis
How many matches were analyzed?	[N eligible matches]
Which team had the most wins?	[Team and win count]
Which team had the highest win rate?	[Team, rate, and matches played]
Did toss winners also win more often?	[Count and percentage]
Was a batting-first / chasing pattern visible?	[Result, denominator, and caveat]
Did outcomes differ by season or venue?	[Main observed pattern]

Interpretation guidelines

• Always include denominators and percentages, not only counts.

• Avoid ranking teams by win percentage when their match counts are very small without noting that limitation.

• Separate wins by runs from wins by wickets; these margins have different meanings.

• Treat seasons and venues as context variables; competition formats and conditions can vary.

• State clearly that the analysis identifies patterns in historical records and is not a reliable guarantee of future match outcomes.

8. Conclusion & Future Scope

This project proposes a Python-based workflow for analyzing cricket match outcomes using structured match-level records. Pandas supports data loading and cleaning, basic statistics summarize team and season performance, and visualizations help compare win rates, toss outcomes, and innings strategies where the required fields are available. The final conclusion must be updated with verified outputs from the selected dataset.

Limitations

• No match dataset was supplied with the original report, so numerical findings in the illustrative charts are not factual results.

• Public datasets may contain inconsistent team names, missing winners, revised season labels, or different definitions of match result.

• Historical association does not prove causation and should not be used alone to predict an individual match.

• Win rate comparisons can be affected by unequal match counts, changing tournament formats, and team composition.

Future scope

• Use verified IPL match data and document the exact source URL, license, and access date.

• Add ball-by-ball data to study powerplay, middle-over, and death-over performance.

• Compare performance by venue, season, toss decision, and target size.

• Build a transparent baseline prediction model using train/test splits and suitable evaluation metrics, if sufficient data is available.

• Create an interactive dashboard with filters for season and team.

9. References, Deliverables & Declaration

References

1. ESPNcricinfo. IPL Records and Statistics. Use the relevant official records pages for background and verification.

2. Cricsheet. Open cricket data and match files. Confirm the available formats and license before use.

3. Kaggle. IPL / cricket match datasets. Record the exact dataset title, uploader, URL, and access date for the file actually used.

4. Python Software Foundation. Python Documentation — https://docs.python.org/3/

5. pandas development team. pandas documentation — https://pandas.pydata.org/docs/

6. Matplotlib development team. Matplotlib documentation — https://matplotlib.org/stable/

Project links & deliverables

Deliverable	Team entry / status
Dataset	[Insert exact CSV filename and source]
Notebook (.ipynb)	[Insert Colab / notebook link]
Repository	[Insert GitHub repository link, if available]
Report	Sports_Match_Outcome_Pattern_Analysis_Using_Python.pdf
README	Include setup, data source, column mapping, and run instructions

Declaration by Student(s)

We declare that this project report should be finalized using the dataset and analysis reviewed by our team. All data sources and AI assistance must be acknowledged. We are responsible for checking the code, verifying the results, and explaining the findings.

Tea Team Leader: G Jaswanth Reddy (BETM1AI25159) Signature: _____

Tea Team Member: K Manoj Kumar (BETN1AI25183) Signature: _____

Tea Team Member: K Punith kumar (BETN1AI25176) Signature: _____

Faculty Use Only

Evaluation component	Max marks	Marks awarded
Data Cleaning & Preprocessing	2	
Exploratory Data Analysis	2	
Visualization & Observations	2	
Statistical Analysis	2	
Insights, Interpretation & Conclusion	2	
Total	10	

Faculty Mentor Signature: _____ Date: _____